

# Kotlin抽象类教学讲义（零基础入门）

各位同学，今天咱们重点攻克Kotlin中的“抽象类”——它是面向对象编程中“抽象设计”的核心，就像建筑工程的“通用设计规范”，只规定必须有的部分，具体细节交给不同的建筑去实现。在Android开发中，抽象类常用于封装通用逻辑（比如各种UI组件的基础行为），能大幅提升代码的复用性和规范性。这节课咱们用“比喻+代码实操”的方式，保证零基础也能吃透！

## 一、先搞懂：什么是抽象类？

咱们从生活中的例子入手：假设学校要做一批“校园卡”，不管是学生卡、教师卡还是职工卡，都必须有“卡号、姓名、所属部门”这三个信息，也必须能“刷卡”——但“学生卡刷卡开门”和“教师卡刷卡查课表”的具体逻辑不一样。

这里的“校园卡通用规范”就是**抽象类**：

- 必须有的部分**：卡号、姓名等属性，刷卡等方法（抽象类中叫“抽象属性/方法”）；
- 不确定的部分**：刷卡的具体逻辑（由学生卡、教师卡等“具体类”去实现）；
- 不能直接用**：“通用规范”只是设计，不能直接当成一张能用的卡（抽象类不能实例化，不能new）。

Kotlin中用 `abstract` 关键字定义抽象类，核心作用是“制定标准、封装通用逻辑”，让子类在统一标准下实现具体功能。

## 二、抽象类的核心组成：抽象与非抽象成员

抽象类里可以放两种内容：**抽象成员**（必须由子类实现的“标准”）和**非抽象成员**（子类可以直接用的“通用功能”），就像“校园卡规范”里，“卡号格式验证”是通用功能（非抽象），“刷卡逻辑”是标准（抽象）。

### 1. 抽象成员 vs 非抽象成员

| 类型    | 定义关键字                                           | 特点               | 例子                               |
|-------|-------------------------------------------------|------------------|----------------------------------|
| 抽象属性  | <code>abstract val/var</code>                   | 只有声明，没有赋值，子类必须赋值 | <code>abstract val cardId</code> |
| 抽象方法  | <code>abstract fun</code>                       | 只有声明，没有实现，子类必须实现 | <code>abstract fun swipe</code>  |
| 非抽象属性 | <code>val/var</code> （无 <code>abstract</code> ） |                  |                                  |

|       |                 |                 |                                     |
|-------|-----------------|-----------------|-------------------------------------|
|       |                 | 有具体值，子类可直接用或重写  | val schoolName = 学"                 |
| 非抽象方法 | fun (无abstract) | 有具体实现，子类可直接用或重写 | fun checkCardId<br>cardId.length == |

## 2. 完整案例：定义“校园卡”抽象类

Code block

```
1 // 抽象类：校园卡（用abstract修饰，不能实例化）
2 abstract class CampusCard(
3     // 抽象属性：卡号（子类必须赋值）
4     abstract val cardId: String,
5     // 抽象属性：姓名（子类必须赋值）
6     abstract val userName: String
7 ) {
8     // 非抽象属性：所属学校（所有校园卡都一样，直接赋值）
9     val schoolName: String = "阳光大学"
10
11     // 抽象方法：刷卡（子类必须实现具体逻辑）
12     abstract fun swipeCard()
13
14     // 非抽象方法：验证卡号格式（通用功能，子类可直接用）
15     fun validateCard(): Boolean {
16         // 调用抽象属性cardId（虽然现在不知道值，但子类会实现）
17         return cardId.startsWith("Y") && cardId.length == 10
18     }
19 }
```

注意：如果尝试 `val card = CampusCard("123", "小明")` 会直接报错——抽象类不能创建对象，必须通过子类“落地”才能用。

## 三、抽象类的使用：继承与实现

抽象类的“落地”靠**子类继承**，子类必须遵守两个规则：

- 1. 用 `:` 继承抽象类（Kotlin单继承，一个类只能继承一个抽象类）；
- 2. 必须实现抽象类中所有的抽象属性和抽象方法（用 `override` 修饰）。

### 1. 子类实现：学生卡与教师卡

Code block

```
1 // 子类1：学生卡（继承校园卡抽象类）
```

```

2  class StudentCard(
3      // 实现抽象属性cardId
4      override val cardId: String,
5      // 实现抽象属性userName
6      override val userName: String,
7      // 学生卡特有属性：学号
8      val studentId: String
9  ) : CampusCard(cardId, userName) {
10
11      // 实现抽象方法swipeCard（学生卡的刷卡逻辑）
12      override fun swipeCard() {
13          println("学生 $userName 刷卡成功，可进入教学楼和宿舍")
14      }
15
16      // 学生卡特有方法：查成绩
17      fun checkScore() {
18          println("$userName 的期末成绩已更新")
19      }
20  }
21
22  // 子类2：教师卡（继承校园卡抽象类）
23  class TeacherCard(
24      override val cardId: String,
25      override val userName: String,
26      // 教师卡特有属性：工号
27      val teacherId: String
28  ) : CampusCard(cardId, userName) {
29
30      // 实现抽象方法swipeCard（教师卡的刷卡逻辑）
31      override fun swipeCard() {
32          println("教师 $userName 刷卡成功，可进入办公室和实验室")
33      }
34
35      // 重写抽象类的非抽象方法（可选）
36      override fun validateCard(): Boolean {
37          // 教师卡号规则和学生卡不同，重写验证逻辑
38          return cardId.startsWith("T") && cardId.length == 8
39      }
40  }

```

## 2. 调用子类：抽象类的“落地”使用

子类是具体类，可以正常创建对象并调用方法，既可以用自己的特有功能，也能用上抽象类的通用功能：

Code block

```
1 fun main() {
2     // 创建学生卡对象
3     val studentCard = StudentCard("Y202400001", "小明", "202401001")
4     // 调用抽象类的非抽象方法
5     println("学生卡验证: ${if (studentCard.validateCard()) "通过" else "失败"}")
6     // 调用实现的抽象方法
7     studentCard.swipeCard()
8     // 调用学生卡特有方法
9     studentCard.checkScore()
10
11     println("-----")
12
13     // 创建教师卡对象
14     val teacherCard = TeacherCard("T202401", "李老师", "G2024001")
15     println("教师卡验证: ${if (teacherCard.validateCard()) "通过" else "失败"}")
16     teacherCard.swipeCard()
17 }
```

## 执行结果与逻辑分析

Code block

```
1  学生卡验证：通过
2  学生 小明 刷卡成功，可进入教学楼和宿舍
3  小明 的期末成绩已更新
4  -----
5  教师卡验证：通过
6  教师 李老师 刷卡成功，可进入办公室和实验室
```

核心价值体现：

- 抽象类制定了“必须有卡号、能刷卡”的标准，保证所有校园卡都符合规范；
- 子类各自实现刷卡逻辑，保留了灵活性；
- 通用功能（学校名称、卡号验证）放在抽象类，避免重复代码。

## 四、open关键字：让类和成员可被继承/重写

在学习抽象类时，很多同学会疑惑：“为什么普通类的方法不能直接重写？”这就涉及到Kotlin的一个核心设计——**默认不可继承/重写**。为了保证代码安全性，Kotlin中所有普通类、属性和方法默认都是“封闭”的，必须用 `open` 关键字标记，才能被子类继承或重写。

而抽象类和抽象成员之所以能被继承/实现，是因为它们隐式包含了 `open` 的特性，无需额外标记。下面我们详细拆解open关键字的使用场景。

## 1. 核心规则：默认封闭，open解锁

Kotlin的设计哲学是“明确优于隐晦”，所以：

- 普通类默认不能被继承，加 `open` 后才能成为“可继承类”；
- 普通属性/方法默认不能被重写，加 `open` 后才能被子类重写；
- 抽象类/抽象成员默认可继承/实现，无需加 `open`；
- 子类重写时必须用 `override` 关键字，不可省略。

## 2. 实战对比：无open vs 有open的类

### 错误案例：普通类默认不可继承

Code block

```
1  // 普通类：默认无open，不可继承
2  class NormalClass {
3      fun normalMethod() {
4          println("普通方法，默认不可重写")
5      }
6  }
7
8  // 错误：NormalClass未加open，不能被继承
9  // class SubClass : NormalClass() { }
10 // 错误：normalMethod未加open，不能被重写
11 // class NormalClass2 {
12 //     override fun normalMethod() { }
13 // }
```

### 正确案例：open类可继承，open方法可重写

Code block

```
1  // 1. 加open的类：可被继承
2  open class OpenClass {
3      // 2. 加open的方法：可被重写
4      open fun openMethod() {
5          println("open方法，可被重写")
6      }
7
8      // 无open的方法：不可被重写
9      fun closeMethod() {
10         println("无open方法，不可被重写")
11     }
```

```
12
13     // 加open的属性：可被重写
14     open val openProp = "open属性"
15 }
16
17 // 子类继承open类
18 class SubClass : OpenClass() {
19     // 重写open方法，必须加override
20     override fun openMethod() {
21         super.openMethod() // 可选：调用父类实现
22         println("子类重写了open方法")
23     }
24
25     // 重写open属性，必须加override
26     override val openProp = "子类重写后的open属性"
27
28     // 错误：closeMethod无open，不能重写
29     // override fun closeMethod() { }
30 }
31
32 fun main() {
33     val sub = SubClass()
34     sub.openMethod()
35     println(sub.openProp)
36     sub.closeMethod()
37 }
```

执行结果

Code block

```
1  open方法，可被重写
2  子类重写了open方法
3  子类重写后的open属性
4  无open方法，不可被重写
```

3. open与抽象类的关系：互补的设计

抽象类和open类都是为了“代码复用和扩展”，但适用场景不同，二者是互补关系：

| 类型            | 核心特点                   | 使用场景                          |
|---------------|------------------------|-------------------------------|
| 抽象类（abstract） | 隐式open，有未实现的抽象成员，不能实例化 | 制定标准，要求子类必须实现特定功能（如CampusCard |

|             |                      |                               |
|-------------|----------------------|-------------------------------|
|             |                      | 的swipeCard)                   |
| open类（open） | 显式open，所有成员都已实现，可实例化 | 提供完整功能，允许子类根据需求重写部分功能（如基础工具类） |

## 案例：抽象类与open类的结合使用

在Android开发中，我们常将抽象类作为“基类标准”，open类作为“通用实现”，比如：

Code block

```
1 // 抽象类：定义网络请求的标准
2 abstract class BaseRequest {
3     // 抽象方法：必须由子类实现请求URL
4     abstract fun getUrl(): String
5
6     // 非抽象方法：通用的请求头配置（隐式open，可重写）
7     open fun getHeaders(): Map<String, String> {
8         return mapOf("Content-Type" to "application/json")
9     }
10
11    // 通用请求逻辑（不可重写，无open）
12    fun execute() {
13        println("请求URL: ${getUrl()}")
14        println("请求头: ${getHeaders()}")
15        println("执行网络请求...")
16    }
17 }
18
19 // open类：通用的GET请求实现（可被继承）
20 open class GetRequest : BaseRequest() {
21     override fun getUrl(): String {
22         return "https://api.example.com/get"
23     }
24
25    // 可选：重写请求头
26    override fun getHeaders(): Map<String, String> {
27        val baseHeaders = super.getHeaders()
28        return baseHeaders + mapOf("Method" to "GET")
29    }
30 }
31
32 // 子类：具体的用户信息请求（继承open类）
33 class UserInfoRequest : GetRequest() {
34     // 重写URL，获取用户信息
35     override fun getUrl(): String {
```

```
36         return "https://api.example.com/get/user"
37     }
38 }
39
40 fun main() {
41     val userRequest = UserInfoRequest()
42     userRequest.execute()
43 }
```

## 执行结果

Code block

```
1  请求URL: https://api.example.com/get/user
2  请求头: {Content-Type=application/json, Method=GET}
3  执行网络请求...
```

## 4. Android开发中的open使用场景

open关键字在Android开发中无处不在，最典型的的就是**Activity/Fragment的继承**：

Code block

```
1  // Android中的Activity就是open类（源码中定义为open class Activity）
2  class MainActivity : AppCompatActivity() {
3      // 重写Activity的open方法（如onCreate）
4      override fun onCreate(savedInstanceState: Bundle?) {
5          super.onCreate(savedInstanceState)
6          setContentView(R.layout.activity_main)
7      }
8
9      // 重写Activity的open方法（如onStart）
10     override fun onStart() {
11         super.onStart()
12         println("Activity已启动")
13     }
14 }
```

如果Activity没有加open，我们就无法创建自己的MainActivity继承它——这就是open关键字的核心作用：为代码扩展打开“入口”。

## 五、抽象类在Android开发中的实战场景

在Android中，抽象类的使用非常广泛，最典型的就是“封装通用UI逻辑”和“定义回调接口的替代方案”。下面看两个真实开发案例：

## 1. 场景1：封装通用按钮逻辑

APP中所有按钮都需要“点击防重复、加载状态显示”的通用逻辑，用抽象类封装后，具体按钮只需实现点击后的业务逻辑：

Code block

```
1  // 抽象类：通用功能按钮
2  abstract class BaseFunctionButton(context: Context) : Button(context) {
3      // 非抽象属性：是否正在加载
4      private var isLoading = false
5
6      init {
7          // 通用点击事件（防重复点击）
8          setOnClickListener {
9              if (!isLoading) {
10                 isLoading = true
11                 showLoading()
12                 // 抽象方法：具体业务逻辑交给子类
13                 onClick()
14                 // 模拟加载完成
15                 Handler(Looper.getMainLooper()).postDelayed({
16                     isLoading = false
17                     hideLoading()
18                 }, 1000)
19             }
20         }
21     }
22
23     // 非抽象方法：显示加载状态
24     private fun showLoading() {
25         text = "加载中..."
26         isEnabled = false
27     }
28
29     // 非抽象方法：隐藏加载状态
30     private fun hideLoading() {
31         text = getButtonText()
32         isEnabled = true
33     }
34
35     // 抽象方法1：获取按钮文本（子类定义）
36     abstract fun getButtonText(): String
37 }
```

```

38     // 抽象方法2: 具体点击业务 (子类实现)
39     abstract fun onClick()
40 }
41
42 // 子类: 登录按钮 (实现通用按钮)
43 class LoginButton(context: Context) : BaseFunctionButton(context) {
44     override fun getButtonText(): String {
45         return "登录"
46     }
47
48     override fun onClick() {
49         // 登录的具体业务逻辑
50         println("执行登录请求, 验证账号密码")
51     }
52 }
53
54 // 子类: 注册按钮 (实现通用按钮)
55 class RegisterButton(context: Context) : BaseFunctionButton(context) {
56     override fun getButtonText(): String {
57         return "注册"
58     }
59
60     override fun onClick() {
61         // 注册的具体业务逻辑
62         println("执行注册请求, 创建新账号")
63     }
64 }

```

## 2. 场景2: 列表数据加载的基类

RecyclerView加载数据时, 都需要“加载中、加载成功、加载失败”的状态管理, 用抽象类封装通用逻辑:

Code block

```

1 // 抽象类: 列表数据加载基类
2 abstract class BaseListLoader<T> {
3     // 加载状态
4     enum class LoadState { LOADING, SUCCESS, FAIL }
5
6     // 非抽象方法: 通用加载流程
7     fun loadData() {
8         updateState(LoadState.LOADING)
9         // 抽象方法: 具体加载逻辑 (网络/本地) 交给子类
10        doLoadData(object : LoadCallback<T> {
11            override fun onSuccess(data: List<T>) {

```

```
12         updateState(LoadState.SUCCESS)
13         // 抽象方法：数据刷新逻辑交给子类
14         refreshUI(data)
15     }
16
17     override fun onFail(errorMsg: String) {
18         updateState(LoadState.FAIL)
19         showError(errorMsg)
20     }
21 })
22 }
23
24 // 非抽象方法：显示错误信息（通用）
25 private fun showError(errorMsg: String) {
26     println("加载失败: $errorMsg")
27 }
28
29 // 抽象方法1：具体加载逻辑（子类实现）
30 abstract fun doLoadData(callback: LoadCallback<T>)
31
32 // 抽象方法2：刷新UI（子类实现）
33 abstract fun refreshUI(data: List<T>)
34
35 // 抽象方法3：更新状态（子类实现，比如显示加载动画）
36 abstract fun updateState(state: LoadState)
37
38 // 回调接口：通知加载结果
39 interface LoadCallback<T> {
40     fun onSuccess(data: List<T>)
41     fun onFail(errorMsg: String)
42 }
43
44
45 // 子类：商品列表加载器（实现基类）
46 class GoodsListLoader : BaseListLoader<String>() {
47     override fun doLoadData(callback: BaseListLoader.LoadCallback<String>) {
48         // 模拟网络请求加载商品数据
49         val isSuccess = true
50         if (isSuccess) {
51             val goodsList = listOf("手机", "电脑", "平板")
52             callback.onSuccess(goodsList)
53         } else {
54             callback.onFail("网络连接超时")
55         }
56     }
57
58     override fun refreshUI(data: List<String>) {
```

```
59         // 刷新RecyclerView的UI
60         println("商品列表更新，共${data.size}件商品：$data")
61     }
62
63     override fun updateState(state: BaseListLoader.LoadState) {
64         // 显示对应的加载状态（比如加载中显示进度条）
65         println("当前加载状态：$state")
66     }
67 }
```

使用时只需调用 `GoodsListLoader().loadData()`，就能自动执行“加载中→请求数据→刷新UI”的完整流程，子类只关注自己的业务细节。

## 五、抽象类 vs 接口：别搞混！

很多小白会把抽象类和接口弄混，它们都能制定标准，但核心区别很大，这里用一张表讲清楚：

| 对比维度  | 抽象类（abstract class）        | 接口（interface）                |
|-------|----------------------------|------------------------------|
| 继承/实现 | 单继承（一个类只能继承一个）             | 多实现（一个类可实现多个）                |
| 属性    | 可包含抽象和非抽象属性（有存储能力）         | 只能包含抽象属性或带默认getter的属性（无存储能力） |
| 方法    | 可包含抽象和非抽象方法                | 可包含抽象方法和默认方法                 |
| 核心作用  | 封装通用逻辑+制定标准（“有肉有骨”）        | 纯制定标准，实现多态和解耦（“只有骨架”）        |
| 使用场景  | 有通用逻辑可复用的场景（如BaseActivity） | 不同类需遵守同一标准的场景（如点击事件接口）       |

💡 记忆口诀：抽象类是“亲爹”（单继承，给你留财产——通用逻辑），接口是“干爹”（多实现，只给你立规矩——标准）。

## 六、总结与课后作业

### 1. 核心知识点回顾

- 抽象类用 `abstract` 修饰，不能实例化，作用是“制定标准+封装通用逻辑”；
- 抽象类包含抽象成员（必须子类实现）和非抽象成员（子类可直接用）；

- 子类用 `open` 继承抽象类，必须实现所有抽象成员，可重写非抽象成员；
- 抽象类单继承，接口多实现，前者侧重复用，后者侧重解耦；
- Kotlin默认不可继承/重写，普通类和成员需加 `open` 才能被继承/重写；
- 抽象类/成员隐式包含 `open` 特性，无需额外标记。

## 2. 课后作业：实现“APP主题切换”抽象类

需求：

- 定义抽象类 `AppTheme`，包含：
  - 抽象属性：`themeName`（主题名称，如“浅色主题”）、`primaryColor`（主色调，String类型）；
- 抽象方法：`applyTheme()`（应用主题的具体逻辑）；
- 非抽象方法：`saveTheme()`（通用逻辑，打印“当前主题已保存：XXX”）。
- 定义两个子类：`LightTheme`（浅色主题）和 `DarkTheme`（深色主题），分别实现抽象属性和方法：
  - 浅色主题：`themeName`=“浅色主题”，`primaryColor`=“#FFFFFF”，`applyTheme()`打印“浅色主题已应用，文字颜色#000000”；
- 深色主题：`themeName`=“深色主题”，`primaryColor`=“#000000”，`applyTheme()`打印“深色主题已应用，文字颜色#FFFFFF”。
- 创建两个主题对象，分别调用`applyTheme()`和`saveTheme()`方法。

（注：文档部分内容可能由 AI 生成）